❑ Exercice 1 : *Intersection*

En OCaml, on représente une partie de $\mathbb{N}$ par une liste *triée* d'entiers, par exemple $[9; 15; 22; 28]$.

1. Écrire une fonction `intersection : int list -> int list -> int list` calculant l'intersection de deux parties (le résultat doit être trié).
2. Prouver la terminaison de `intersection`.

❑ Exercice 2 : *Correction*

1. Écrire en C une fonction de signature `int somme(int tab[], int taille)` renvoyant la somme des éléments d'un tableau.
2. Prouver la correction de votre fonction.

❑ Exercice 3 : *Notation $\mathcal{O}$*

1. Déterminer un grand $\mathcal{O}$ des suites de terme général :
   a) $2023n^2$ b) $n^2 + 10^9 n$ c) $3n + 7\log n$
   d) $2^{n+7} + n^{10}$ e) $\sqrt{19n^2 + 3}$ f) $\log(3n) + \log(n)$
2. Montrer que si $u_n = \mathcal{O}(v_n)$ et $v_n = \mathcal{O}(w_n)$, alors $u_n = \mathcal{O}(w_n)$.
3. Montrer que $\mathcal{O}(u_n) + \mathcal{O}(v_n) = \mathcal{O}(\max(u_n, v_n))$.
4. Montrer que si $u_n = \mathcal{O}(a_n)$ et $v_n = \mathcal{O}(b_n)$, alors $u_n v_n = \mathcal{O}(a_n b_n)$.
5. Déterminer un grand $\mathcal{O}$ (le « meilleur » possible) des expressions suivantes :
   a) $\mathcal{O}(n^4) + \mathcal{O}(n^2)$ b) $\mathcal{O}(n^5) + \mathcal{O}(n^5)$ c) $\mathcal{O}(n^3) + \mathcal{O}(\log(n))$
   d) $\mathcal{O}(n^4) \times \mathcal{O}(n^3)$ e) $\mathcal{O}(n^4) \times \mathcal{O}(\sqrt{n})$ f) $\mathcal{O}(n^2) \times \mathcal{O}(\log(n))$

❑ Exercice 4 : *Multiplier en additionnant*

```
int mult(int n, int p){
    int prod = 0;
    while (p>0){
        prod = prod + n;
        p = p -1;}
    return prod;}
```

1. En supposant $p > 0$, montrer la terminaison de la fonction `mult`.
2. Prouver que cette fonction renvoie bien $p \times n$.
3. Déterminer sa complexité.

❑ Exercice 5 : *Vérification du tri*

1. Écrire un algorithme permettant de vérifier qu'un tableau est trié par ordre croissant.
2. En proposer une implémentation en OCaml permettant de vérifier qu'une liste est triée.
3. Prouver que votre algorithme est correct.
4. Déterminer sa complexité.

❑ Exercice 6 : *Exponentiation rapide*

On rappelle la fonction d'exponentiation rapide dans sa version itérative :

```
float expo(float a, int n){
    float cp = a;
    float res = 1;
    while (n!=0){
        if (n%2==1){
            res = res*cp;}
        cp = cp*cp;
        n=n/2;}
    return res;}
```

1. Prouver que cet algorithme termine.
2. Prouver qu'il est correct.
   ⊗ En notant n_0 la valeur initiale de n, on pourra considérer l'invariant suivant : $\texttt{res} \times \texttt{cp}^n = a^{n_0}$.
3. Donner sa complexité.

❑ Exercice 7 : *Retour sur la multiplication*

On donne la fonction suivante :

```c
int multiplie(int n, int p){
    int prod = 0;
    while (n>0){
        if (n%2==1) {
            prod = prod+p;}
        n= n / 2;
        p = p*2;}
    return prod;}
```

1. Vérifier à la main, sur deux entiers naturels de votre choix, que cette fonction est conforme à sa spécification.
2. Montrer la terminaison de cette fonction.
3. Montrer que cette fonction est bien conforme à sa spécification.
4. Déterminer sa complexité.

❑ Exercice 8 : *Tri à bulles*

1. Rappeler le principe du tri à bulles.
2. En écrire une implémentation en C, dans laquelle on vérifie à chaque passage qu'au moins une inversion a été effectuée. Si tel n'est pas le cas, on termine immédiatement l'algorithme puisque cela signifie que les éléments sont triés.
3. Montrer la terminaison de cette fonction.
4. Prouver qu'elle est correcte.
   ⊗ On pourra exhiber un invariant qui donne le nombre d'éléments figurant à leur place finale après chaque itération.
5. Déterminer la complexité de cet algorithme dans le pire des cas et dans le meilleur des cas.

❑ Exercice 9 : *Recherche dichotomique*

Algorithme : Recherche dichotomique

Entrées : $a \in \mathbb{N}$ et un tableau d'entiers t **trié** de longueur n
Sorties : Booléen indiquant si $a \in t$.

```
d ← 0
f ← n − 1
tant que f − d ⩾ 0 faire
    m ← (f + d)/2
    si t[m] = a alors
        return Vrai
    fin
    si a < t[m] alors
        f ← m − 1
    fin
    sinon
        d ← m + 1
    fin
fin
return Faux
```

1. Étudier la terminaison de cet algorithme.
2. Prouver que cet algorithme est totalement correct.
3. Déterminer sa complexité.

❑ **Exercice 10** : *Nombre de chiffres d'un entier*

1. Écrire en C une version itérative d'une fonction donnant le nombre de chiffres d'un entier naturel.
2. Écrire une version récursive en OCaml.
3. Prouver la terminaison dans les deux cas.
4. Prouver la correction dans les deux cas.
5. Déterminer la complexité dans les deux cas.

❑ **Exercice 11** : *Majorité absolue*

On considère les résultats d'un vote sous la forme d'un tableau d'entiers positifs : lorsqu'on rencontre la valeur `i`, cela signifie que le candidat numéro `i` a obtenu un vote. Par exemple, si le tableau contient les valeurs `[2, 3, 1, 2, 2, 2, 4, 1, 1, 2]`, alors le candidat 1 a obtenu 3 voix, le candidat 2 a obtenu 5 voix, ... On cherche à concevoir un algorithme efficace permettant de déterminer (s'il existe) le candidat ayant obtenu la majorité absolue. Dans le cas où aucun candidat n'a la majorité absolue, l'algorithme doit renvoyer `-1`. On note C le nombre de candidats et N le nombre de votes.

1. Proposer un algorithme naïf permettant de résoudre ce problème et donner sa complexité. On donne ci-dessous **le début** de l'implémentation en C d'un algorithme proposé par R. Boyer et J. S. Moore :

```c
int majorite_absolue(int vote[], int size){
    int nb_votes = 0;
    int candidat = 0;
    for (int i = 0; i < size; i++){
        if (nb_votes == 0){
            candidat = vote[i];
            nb_votes = 1;}
        else{
            if (vote[i] == candidat){
                nb_votes += 1;}
            else{
                nb_votes -= 1;}
        }
    }
```

2. Prouver qu'à la fin de la boucle `for`, la variable `candidat` contient le numéro du seul candidat éventuellement majoritaire.
3. Compléter l'implémentation en ajoutant les lignes permettant de vérifier après la boucle `for` que ce candidat est effectivement majoritaire.
4. Donner la complexité de cet algorithme.

❑ **Exercice 12** : *Fonction 91*

La fonction 91, notée f_{91}, due à l'informaticien J. McCarthy est définie sur $\mathbb{N}$ par :

$$f_{91}(n) = \begin{cases} n-10 & \text{si } n > 100, \\ f_{91}(f_{91}(n+11)) & \text{si } n \leqslant 100. \end{cases}$$

1. Calculer $f_{91}(101)$, $f_{91}(200)$, $f_{91}(2025)$.
2. Écrire une fonction `f91` en OCaml calculant cette fonction.
3. Montrer que si $n \in [\![90;100]\!]$ alors $f_{91}(n) = f_{91}(n+1)$. En déduire la valeur de f_{91} sur $[\![90;100]\!]$.
4. Prouver que, pour tout $n \in [\![0;101]\!]$, cette fonction termine et vaut 91. On pourra considérer l'entier $p \in \mathbb{N}^*$ tel que $90 \leqslant n + 11p \leqslant 100$.

❑ **Exercice 13** : *Équations de récurrence*

Dans l'étude d'algorithmes récursifs, le nombre d'opérations élémentaires $T(n)$ pour une entrée de taille n satisfait souvent une équation de récurrence. On suppose dans tout l'exercice que $T(1) = 1$. Déterminer l'ordre de grandeur asymptotique en notation $\mathcal{O}$ de $T(n)$ dans chacun des cas suivants :

1. $T(n) = T(n-1) + n$ pour $n \geqslant 2$. Quel algorithme de tri usuel correspond à cette équation ?

2. $T(n) = 2T(n-1)+1$ pour $n \geqslant 2$. À quel problème algorithmique classique correspond cette récurrence ?

3. $T(n) = 2T(\lfloor n/2 \rfloor) + n$ pour $n \geqslant 2$. On pourra d'abord expliciter $T(2^k)$ pour tout $k \in \mathbb{N}$, puis en déduire la complexité dans le cas général.
4. $T(n) = 3T(\lfloor n/2 \rfloor)+n$ pour $n \geqslant 2$. Même méthode que précédemment en étudiant $n = 2^k$ (on rappelle que cette récurrence apparaît dans l'algorithme de multiplication rapide d'entiers de Karatsuba).

❑ **Exercice 14** : *Recherche simultanée du minimum et du maximum*

Soit t un tableau d'entiers de taille $n \geqslant 1$. On souhaite déterminer simultanément la valeur minimale et la valeur maximale de t.

1. Combien de comparaisons entre éléments du tableau effectue un algorithme naïf qui recherche successivement le minimum puis le maximum ?
2. On propose d'optimiser le nombre de comparaisons en traitant les éléments par paires :
   — Si n est impair, on initialise le minimum et le maximum courants à $t[0]$, et on examine les éléments restants deux par deux.
   — Si n est pair, on compare $t[0]$ et $t[1]$ pour initialiser le minimum et le maximum courants, et on examine les éléments restants deux par deux.
   — Pour chaque paire $\{t[i], t[i+1]\}$, on compare d'abord ces deux éléments entre eux, puis on compare le plus petit au minimum courant et le plus grand au maximum courant.

   Combien de comparaisons entre éléments du tableau sont effectuées pour chaque paire traitée ?
3. En déduire le nombre total de comparaisons effectuées par cet algorithme en fonction de la parité de n. Montrer que, dans tous les cas, ce nombre vaut $\lceil \frac{3n}{2} \rceil - 2$.
4. Écrire en C une fonction implémentant cet algorithme.
5. Prouver formellement sa correction.